CldOps Cheatsheet

Doriano Di Pierro & Fabio Zahner

1 General Definitions

Definitions:

- **Cloud Operations** is the practice of managing and optimizing cloud-based services and infrastructure.
- **GitOps**: Git-based infrastructure and application deployment; uses Git as single source of truth; enables CI/CD, automation, version control, and declarative configuration.
- **DevOps** combines development and operations; focuses on automation, collaboration, CI/CD, monitoring, and agile delivery.
- **Continuous Integration:** every pushed change (even on dev branches) is automatically built & tested so it keeps passing all checks.
- **Platform Engineering:** building internal self-service platforms (tooling, pipelines, infra) so dev teams ship faster; CI/CD is a key part.

1.1 DevOps Cycle

Plan (add Objectives and Requirements to Backlog), **Code** (add Code to Repo), **Build** (Pipelines runs on push, builds and unit tests software), **Test** (Build is deployed to staging environment, tested using E2E, load, accessibility tests), **Release** (snapshot of code is versioned, changes are documented), **Deploy** (release is installed onto production environment), **Operate** (application should run smoothly, issues are troubleshooted and documented, infrastructure is scaled), **Monitor** (Application Data is gathered and used for planning)

Difference Between Continuous Delivery & Continuous Deployment: Deployment automatically pushes from staging to production, in Delivery this is manual.

CD&D Deployment Strategies:

Rolling Deployment: Update infrastructure gradually, minimal downtime

Blue-Green: Two environments: Old and new versions respectively

Canary: Small user group tests first

Feature Flag: Deploy but activate later, can be toggled

Dark Launching: Rolling out a feature invisible for users, test its performance in the background

2 GitLab Pipelines

```
stages:
  - build
  - test
  - deploy
cache:
  paths:
    - .cache/
build:
  stage: build
  script:
    - echo "Building..."
    - mkdir -p artifacts && echo "artifact" >
        artifacts/output.txt
  artifacts:
    paths:
      - artifacts/
    expire_in: 1 hour
test:
  stage: test
  dependencies:
    - build
  script:
    - cat artifacts/output.txt
deploy_staging:
  stage: deploy
  environment:
    name: staging
    url: https://staging.example.com
    on_stop: stop_staging # Unstages env
  script:
    - cat k8.yaml | envsubst | kubectl apply -f -
  artifacts:
    expire_in: 1 hour
stop_staging:
  stage: deploy
  environment:
    name: staging
    action: stop
  script:
    - echo "Stopping staging"
```

2.1 Jobs, Stages & Rules

`.gitlab-ci.yml` is a YAML DSL mapping the DevOps cycle to a pipeline. A **job** is the smallest unit; it belongs to one **stage** and runs its `script` steps on a runner. Jobs in the same stage run in *parallel*, stages run *sequentially*.
rules: decide *when* a job runs (modern replacement for `only/except`). **needs:** builds a DAG so a job can start before its whole stage finishes.

```
deploy:
  stage: deploy
  needs: [build]
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: on_success
    - when: never
```

2.2 Environments

Describe where the code gets deployed (e.g. Local, Integration, Testing, Staging, Production). Can be linked to a K8 cluster (needs to be set up via GitLab UI):

2.3 Cache & Artifacts

Cache: Shared across *jobs* in different **stages** and even **pipelines** (if configured). Used for speeding up tasks (e.g., restoring dependencies). **Artifacts:** Shared only with *downstream jobs* in later **stages**. Must be explicitly declared as `dependencies` to access in later jobs. Uploaded to GitLab and optionally downloadable.

2.4 Push- vs. Pull-Based Deployments

Push-Based: + Easy to use, + flexible deployment targets, - firewall needs to be opened, - pipeline needs to be adjusted for new environments **Pull-Based:** + no need for open firewall, + better scaling, - agent needs to be installed in every cluster

2.5 GitLab Runner

GitLab Runner executes jobs defined in `.gitlab-ci.yml`. Types: **Shared** (managed by GitLab) vs. **Specific** (self-managed). Can run in various executors: `shell`, `docker`, `kubernetes`, etc. Tagged runners can restrict which jobs they pick up.

2.6 More Keywords & Docker Build

`default:` sets fallback `image`/`before_script` for all jobs. `before_script`/`after_script` wrap every job's `script`. `rules: changes:` runs a job only if listed files changed.
Predefined vars: `$CI_REGISTRY[_IMAGE]`, `$CI_REGISTRY_USER`/`_PASSWORD`, `$CI_COMMIT_BRANCH`, `$CI_PROJECT_DIR`.
Pin images by digest (`img@sha256:..`), not `:latest` (tags can silently move → flaky builds).

```
build-image:
  stage: release
  image: docker:28
  services:
    - name: docker:28-dind # Docker-in-Docker
      alias: docker
  variables:
    IMG: $CI_REGISTRY_IMAGE:latest
  before_script:
    - docker login -u $CI_REGISTRY_USER -p
        $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker build -t $IMG -f Dockerfile.prod .
    - docker push $IMG
  rules:
    - changes: [Dockerfile.prod]
```

3 Terraform

Declarative vs. Imperative: Terraform is *declarative* – you describe the desired end state and TF figures out the steps to get there. *Imperative* (shell scripts, manual CLI calls) instead describes *how* to reach a result step by step.
TF doesn't speak directly with an SDK, but rather Terraform -> Provider -> Client SDK. Diffrent providers enable diffrent platforms (AWS, Azure, Kubernetes, ...). **Resources** (`resource`) are the "whatTTF creates and manages (e.g. an EC2 instance). **Data Sources** (`data`) are read-only lookups of existing infrastructure used as input (e.g. an existing AMI or VPC). A sample in HCL (Hashicorp Configuration Language):

```
variable "instance_type" {
  default = "t2.micro"
}
provider "aws" {
  region = "us-east-1"
}
resource "aws_instance" "web" {
  ami = "ami-0c55b159cbfafe1f0"
  instance_type = var.instance_type
}
output "public_ip" {
  value = aws_instance.web.public_ip
}
```

To deploy infrastructure, write HCL in files like `main.tf`, then run `terraform init`, `terraform plan` to show changes that would be made, `terraform apply` to actually apply the changes. Use `terraform destroy` to delete all made changes.

3.1 State

Terraform stores state in the `terraform.tfstate` file, mapping desired (HCL) to actual (real) infrastructure. When working in teams, this state file also has to be shared as the terraform command relies on is validity.
Remote backend: store state remotely (e.g. an S3 Bucket) so the whole team shares it. **State locking** prevents two people running `apply` at once and corrupting the state; the S3 backend locks natively via `use_lockfile` (older setups used a separate DynamoDB table).

```
terraform {
  backend "s3" {
    bucket = "my-tfstate"
    key = "prod/terraform.tfstate"
    region = "us-east-1"
    use_lockfile = true # S3-native state locking
          (replaces DynamoDB)
  }
}
```

3.2 Modules

Modules are reusable, parameterized templates (a folder of `.tf` files) to standardize and DRY up infrastructure. The root config is itself a module and can call child modules:

```
module "vpc" {
  source = "./modules/vpc" # or registry/git URL
  cidr = "10.0.0.0/16" # input variable
}
# reference an output: module.vpc.vpc_id
```

3.3 More Commands & Patterns

`init` does three things: configure the backend, download providers, download modules. `fmt` formats, `validate` checks, `console` opens a REPL (print with `var.x`). `state list` lists tracked resources, `state show <addr>` shows their attributes. Reference any attribute as `<type>.<name>.<attr>` (e.g. `aws_instance.web.public_ip`).

```
import { # adopt existing infra into state
  to = aws_instance.web
  id = "i-0abc123" # provider-side ID; then:
        apply
} # CLI: terraform import aws_instance.web i-0
      abc123

variable "AMIS" { # AMIs differ per region -> use
      a map
  type = map(string)
  default = { us-east-1 = "ami-05b1..." }
}
ami = var.AMIS[var.region] # select AMI for the
      region
```

4 Ansible

Ansible can be used to provision servers. It does not have statefiles and is idempotent, meaning it won't make changes unless it has to. Ansible requires python to be installed on target machines as well as control machines (host).
Terraform vs. Ansible: Terraform *provisions* infrastructure (create VMs, networks); Ansible *configures* the provisioned machines (install & configure software, deploy code). Often combined.
Ansible is *agentless* & *push-based*: over SSH it copies the generated Python module to a temp dir (`~/.ansible/tmp`) on the target, runs it with the remote Python, returns the result as JSON on stdout, then deletes it, nothing stays installed.

4.1 Infrastructure

In a network of servers, one server is the **host**. The host can connect to other machines using SSH. On the host, playbooks can be written in `yaml` files. Run a playbook by using `ansible-playbook playbook.yaml`
Inventory: lists the managed hosts, grouped, in INI or YAML format. Referenced by a play's `hosts:` field.

```
[web]
web1.example.com
web2.example.com ansible_host=10.0.0.5

[web:vars]
ansible_user=deploy
```

```
- name: Example Playbook
  hosts: web
  become: true # run tasks as root (sudo)
  gather_facts: false # skip the implicit setup/
        fact gathering (faster)
  vars:
    packages:
      - nginx
      - curl
    enable_service: true
    secret_password: "{{ vault_password }}"
  roles:
    - myrole
  tasks:
    - name: Install packages
      apt:
        name: "{{ item }}"
        state: present
      loop: "{{ packages }}"
      notify: restart nginx
    - name: Configure app if enabled
      template:
        src: app.conf.j2
        dest: /etc/app.conf
      when: enable_service
      tags: config
  handlers:
    - name: restart nginx
      service:
        name: nginx
        state: restarted
```

4.2 Vaults

Vaults can be used to encrypt data: The file `vault.yaml` with the contents `vault_password: mmysecret"` can be encrypted using `ansible-vault encrypt vault.yml` and then included in a play: `ansible-playbook playbook.yml -ask-vault-pass` To create a file, use `ansible-vault create foo.yaml`
Once encrypted the file is just ciphertext, so committing it to Git is safe & intended – only the vault password must stay out of the repo.

4.3 Collections, Roles & Tags

Collections are bundles of plugins, roles and modules. Install them using `ansible-galaxy collection install <name>`, or define a requirements.yaml to install multiple collections at once. **Roles** are an abstraction above playbooks, allowing to reuse configuration steps: create a role using `ansible-galaxy init <name>`, then use a role like in the example above. **Tags** can be used to execute a subset of tasks instead of the whole playbook. Run only specific tags by appending `-tags <name>` at the end of the ansible-playbook command. There are also two special commands: Tag *always* runs every time, except when explicitly skipped: `-skip-tags=always`. Tag *never* does not run unless specified with `-tags=never`

4.4 Ad-hoc, Modules & Error Handling

Ad-hoc (one module, no playbook): `ansible -i inv -m <module> -a '<args>' <target>`. The `setup` module gathers *facts* (what "Gathering Factsrruns before tasks). `command` vs `shell`: `shell` allows pipes/redirects/vars; *neither is idempotent*. `ansible-inventory -i inv -list` verifies the inventory; `-check` is a dry run. A playbook stops at the first failed task unless `ignore_errors: true`.

```
# ansible -i inv -m ping all
# ansible -i inv -m setup all # gather facts
- name: validate config
  command: httpd -t
  register: out # capture result into a var
  changed_when: out.failed
  ignore_errors: true
- fail: { msg: "bad config" }
  when: out.failed
- file: { path: /data, state: directory } # state
      : file|link|touch|absent
- lineinfile: { path: /etc/x.conf, line: "Include
      sites/*" }
```

4.5 Jinja2

Jinja2 is the templating engine which is used by Ansible. It is used to generate configuration files. Loop over an inventory group inside a `.j2` template (e.g. to build a load-balancer config); `{{ inventory_hostname }}` is the current target.

```
{% for host in groups['webservers'] %}
BalancerMember http://{{ host }}
{% endfor %}
```

5 Kubernetes (K8)

K8 Objects: Persistent entities which signal an intent (e.g. for something to be created on the cluster)
K8 Controller: Tracks an Object and is responsible for bringing the current State closer to the desired State.

Pod: Represents a process running on your cluster. Should contain one container, multiple are possible.
Sidecars: Sidecars are containers that run along the primary container in the same pod. Example use cases might be logging, security, data synchronization.
Init Containers: Similar to sidecars, but run and finish before app containers.
Volume: Assigns physical Storage to a Pod. **Ingress:** Provides external Access to a Service.
ReplicaSet: Makes sure that a specified number of replica pods are running. In practice, deployments are used.
Deployment: Allows to manage one or multiple Pods.
Service: API Resource to expose logical set of Pods in the namespace. Acts as a load balancer (round-robin).
DaemonSet: Ensures that all (or some) Nodes run a copy of a pod. This can be used for running supporting applications (logs, storage)
ConfigMap: Stores configuration data as key-value pairs. Used to decouple config from code. Mounted into pods as env vars or files. Not meant for sensitive data (use Secrets instead).
Secret: Like a ConfigMap but for sensitive data (passwords, tokens, certs). Values are only base64-encoded, *not* encrypted by default (enable encryption at rest / use an external secret store). Mounted as env vars or files.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
replicas: 3 # automatically deploys replicaset
selector:
  matchLabels:
    app: web
template:
  metadata: # pod
    labels:
      app: web
  spec:
    strategy:
      type: RollingUpdate # rolling update
      rollingUpdate:
        maxUnavailable: 1
        maxSurge: 1
    initContainers:
    - name: init
      image: busybox
      command: ['sh', '-c', 'sleep 10']
    containers: # container
    - name: web
      image: nginx
      ports:
      - containerPort: 80
    - name: sidecar
      image: busybox
      command: ['sh', '-c', 'while true; do sleep
            30; done']
```

5.1 Services

A Service gives a stable virtual IP + DNS name to a dynamic set of Pods (selected by labels) and load-balances across them (round-robin). Types:
ClusterIP: (default) internal-only IP, reachable inside the cluster.
NodePort: opens a static port on every node, exposing the Service via `<NodeIP>:<port>`.
LoadBalancer: provisions an external (cloud) load balancer with a public IP that routes to the Service.
Ingress routes external HTTP(S) by host/path to Services (single entrypoint, needs an ingress controller). The newer **Gateway API** (`Gateway` + `HTTPRoute`) replaces Ingress with a more expressive, role-oriented model.
Port trap: Service `port` (its own) vs `targetPort` (the container) vs `nodePort`; an Ingress/HTTPRoute backend must reference the *Service* port, not the container port.

5.2 Using ConfigMaps & Secrets

Inject into Pods as a single env var (`configMapKeyRef`/`secretKeyRef`), all keys at once (`envFrom`), or mounted as files (volume).

```
env:
- name: APIKEY
  valueFrom:
    configMapKeyRef: { name: cfg, key: api-key }
envFrom:
- configMapRef: { name: cfg } # import every key
      as env var
volumeMounts:
- { name: v, mountPath: /etc/cfg }
volumes:
- name: v
  configMap: { name: cfg } # or: secret: {
        secretName: s }
```

5.3 LimitRange & ResourceQuota

LimitRange: min/max + default requests/limits *per container* in a namespace; a Pod outside the range is *rejected*, and a container omitting requests/-limits gets the defaults auto-injected.
ResourceQuota: caps the *summed* resources of a *whole namespace*; a Pod that would exceed the total is rejected.

5.4 Namespaces

Used to separate resources. Only resources in same namespace can communicate directly.

5.5 Rolling Updates

Rolling updates can be used in order to ensure that enough pods are always running. Rolling updates can be using maxUnavailable (maximum No. of Pods upgrading at the same time) and maxSurge (max No. of Pods allowed to run beyond specified No. of replica)

5.6 Scheduling

The `kube-scheduler` determines which nodes run which Pods. We can influence this decision process:

```
kind: Pod
spec:
  nodeSelector:
    disktype: ssd # this label needs to be in pod.
        spec
```

To evaluate if a Node is eligible to run a Pod, the following things are considered: Port availability, CPU & Memory resources, available volumes, specified labels. Additionally, scoring is used to evaluate remaining nodes with criteria: pods of same service should be on different nodes, nodes with few used resources are prioritized, node affinity.
Taints can also be applied on nodes and pods, pods won't be deployed on nodes with matching taints. *Tolerations* can be used to make exceptions to taints. Types of taints: *NoSchedule, PreferNoSchedule, NoExecute*
CLI: `kubectl taint nodes <n> gpu=true:NoSchedule` and `kubectl label nodes <n> disktype=ssd` (remove either by appending a trailing `-`). *Gotcha:* a toleration only *allows* a Pod onto a tainted node, it does not *force* it there – pair it with a `nodeSelector`/affinity to actually pin the Pod.

```
tolerations:
- { key: "gpu", operator: "Equal", value: "true",
      effect: "NoSchedule" }
```

5.7 Probes & Resources

The kubelet uses probes (`httpGet`, `tcpSocket` or `exec`) to check container health:
Liveness: is it alive? On failure → restart the container.
Readiness: is it ready to serve? On failure → remove pod from Service endpoints (no traffic), but don't restart.
Startup: guards slow-starting apps; disables the other probes until it succeeds. Repeated liveness failures → container restarted with exponential back-off → `CrashLoopBackOff`.
requests: guaranteed minimum the scheduler reserves (used for placement).
limits: hard ceiling – CPU is throttled, memory over-limit → OOMKilled.

```
resources:
  requests: { cpu: "250m", memory: "64Mi" }
  limits: { cpu: "500m", memory: "128Mi" }
livenessProbe:
  httpGet: { path: /healthz, port: 80 }
  initialDelaySeconds: 5
  periodSeconds: 10
readinessProbe:
  httpGet: { path: /ready, port: 80 }
```

5.8 Commands

Apply a manifest.yaml: `kubectl {create|apply|replace} -f manifest.yaml`
Generate YAML w/o applying: `kubectl create deploy web -image=nginx -dry-run=client -o yaml > d.yaml`
Imperative: `kubectl run`, `kubectl scale deploy x -replicas=4`, `kubectl expose`
Inspect: `kubectl get pods -A -o wide`, `describe node <n>`, `get nodes -show-labels`
Connecting to a Pod: `kubectl exec -it nginx-xxx - sh` (drop `-it` for one-shot)
Rollout: `kubectl rollout {status|history} deploy x`, `rollout undo deploy x -to-revision=N`
Default namespace: `kubectl config set-context -current -namespace=lab`

6 Helm

A package manager for K8, enabling to reuse configurations for common use cases (DB, monitoring). Helm provides *Charts*, which are a collection of yaml files describing different K8 Objects. When deploying a chart on your cluster, it is called a *Release*. Charts are available through different *Repositories*.
Helm charts use templating (like `{{Release.Name}}`, for information about package, or `{{.Values.xyz.abc | default ëxample"}}` for information passed by values.yaml or via commandline `-set`)

6.1 Commands

```
# Repo commands
helm repo list
helm repo add <repo> <url>
helm repo rm <repo>

helm list # list installed releases
helm install -f values.yaml <release> <chart> #
      install with custom values
helm show <chart | readme | values | all> <
      chartname | repo>

helm upgrade <release> <chart>
helm rollback <release> <revision>

helm template <name> <chart> # render locally, no
      install
helm template . -f values.yaml --set k=v | wc -l
helm lint <chart> # check for errors
helm uninstall <release>
helm history <release> # upgrade & rollback each
      add a revision
helm get all|values|notes <release> # values =
      only overrides
helm search repo <repo>/<chart> -l # list ALL
      versions
helm install x <chart> --version 1.2.3
```

6.2 Chart Structure

`helm create <name>` scaffolds a chart:

- `Chart.yaml` – metadata (name, version, dependencies)
- `values.yaml` – default values (overridable via `-f`/`-set`)
- `templates/` – templated K8 manifests (Go templating)
- `charts/` – bundled subcharts (dependencies)

Dependencies (other charts, e.g. a DB) are declared in `Chart.yaml` and pulled with `helm dependency update`.

6.3 Templating

Named templates live in `templates/_helpers.tpl` (`define` + `include`). Built-ins: `.Chart.Name` (from Chart.yaml), `.Release.Namespace`/`.IsInstall`,

`.Files`. `templates/NOTES.txt` is rendered & printed after install. `helm list` shows CHART version (the package) vs APP VERSION (the software).

```
{{- define "app.name" -}}
{{ default .Chart.Name .Values.name | trunc 63 }}
        # fallback + 63-char cap
{{- end }}
# use it: {{ include "app.name" . }}
resources:
  {{- toYaml .Values.resources | nindent 4 }} #
        inject a whole sub-tree
```

7 Kustomize

The kustomize.yaml defines the resources and transformations. *Transformers* transform your manifests by allowing common fields in the resources to be specified in one place.
In case you want multiple variations on a common resource, the files in `base/` can declare common elements and the files in `overlays/` will declare differences.
Patches allow you to change (patch) already set values.
Traditionally, when editing ConfigMaps or Secrets, the deployment did not change, therefore no pods were restarted. **Generators** allow to define values in the `kustomize.yaml` file, where changes get detected and pods automatically restarted.

```
# A sample kustomize.yaml file
apiVersion: kustomize.config.k8s.io/v1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
# Transformers:
namePrefix: dev-
namespace: my-namespace
commonLabels:
  app: my-app
# Patch:
patches:
- patch: |-
    - op: replace
      path: /metadata/name
      value: nginx-server
  target:
    kind: Service
    name: nginx-app
# Generator:
configMapGenerator:
  - name: app-config
    literals:
      - LOG_LEVEL=debug
# Adding component from component example below
components:
- ../../components/mysql
```

7.1 Components

Components encapsulate a set of modifications. Below is an example component which adds a mysql database. This component can then be included in overlays, base directories (see above) or other components.

```
# Sample component file. Path: components/mysql/
      kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1alpha1
kind: Component # not Kustomization
resources: # resource files are also under /mysql
      /
- deployment.yaml
- service.yaml
```

7.2 Commands

```
kustomize build <dir> # Renders manifests.
kubectl apply -k <dir> # Applies kustomization
      via kubectl.
```

7.3 Images & Overlays

The **images** transformer changes a container's tag/name without touching the Deployment. `configMapGenerator`/`secretGenerator` can also read from `files:` (not just `literals:`) to keep config in one place. An overlay pulls in the shared base via `resources: [../base]`.

```
images:
- name: myrepo/app # existing image (no tag)
  newTag: "0.2.0" # or newName: otherrepo/app
configMapGenerator:
- name: env-cfg
  files: [config.env]
```

Render without applying: `kubectl kustomize <dir>`. Tear down: `kubectl delete -k <dir>`.

7.4 Helm vs. Kustomize

Helm: templating + package manager (variables via `values.yaml`, releases, repos, rollbacks); good for distributing reusable apps.
Kustomize: template-free overlays (patch/merge plain YAML per environment), built into `kubectl (-k)`; good for own manifests across dev/test/prod.
The two are often combined (Helm to package, Kustomize to patch).

8 K8s Monitoring & Logging

Three pillars of observability: *Metrics* (quantitative numbers over time), *Logs* (qualitative event records), *Traces* (follow one request across services via *spans* to find latency/bottlenecks, e.g. Tempo, Jaeger).
Monitoring: metrics, alerts, trends – quantitative (resource usage, request counts, error rates)
Logging: detailed event records – qualitative (stack traces, system messages, business events)
Kubernetes does not persist metrics out of the box. `kubectl logs` only persists logs for running containers, logs lost when container dies. **Agent-Based Monitoring/Logging:** A push-based system where software installed on server collects data and sends it to a central monitoring server. **Agentless Monitoring/Logging:** A central monitoring server requests data from the devices being monitored.

> **Example Stack:** *Monitoring:* Exporters (custom for each app, exposes metrics), Prometheus (metrics collection), Grafana (dashboard), Alertmanager (rule-based alerting) *Logging:* Fluent bit/fluentd (log collection), Loki (aggregation), Grafana (visualization) *Alternative: EFK (see subsection EFK)*

8.1 Monitoring

Prometheus: Written in Go, stores data in a time-series DB. Scrapes metric endpoints over HTTP. Has client libraries to instrument custom apps written in Go, Java, Python, Node.js, .NET, ... Uses PromQL language to aggragate data across labels. Some PromQL examples:

```
sum by (namespace) (kube_pod_info) -- count of
      pods per cluster and namespace
sum by (namespace) (kube_pod_status_ready{
      condition="false"}) -- count of pods not
      ready by namespace
rate(mysql_global_status_commands_total{command=~
      "(select)"}[5m]) * 100 > 35 -- Rate of
      MySQL SELECT commands over last 5 minutes
-- The > 35 can be used to only display outliers
      (threshold/alert conditions)
```

Grafana: Visualization tool for metrics & logs. Connects to data sources (Prometheus, Loki, Elasticsearch)

8.2 Prometheus Operator

`kube-prometheus-stack` bundles operator + Prometheus + Grafana + Alertmanager. The operator watches CRDs (`Prometheus`, `ServiceMonitor`, `PodMonitor`) and turns a `Prometheus` CR into a StatefulSet. A **ServiceMonitor** wires a Service into Prometheus with no config edit – match chain: Prometheus `serviceMonitorSelector` → ServiceMonitor `selector` → Service `selector` → Pods.
node-exporter: hardware/OS metrics.
kube-state-metrics: state of K8 objects, e.g.:

```
kube_deployment_status_replicas_ready{namespace="
      x",deployment="y"}
```

CPU units: `100m` = 0.1 CPU, absolute (same amount on a 1- or 48-core node). Install `metrics-server` for `kubectl top nodes/pods`.

8.3 Logging

Node-Level vs Application Logging: With application logging, the container of the app itself is directly communicating with the Logging Backend. In Node-Level logging, the application writes to a log-file, which the logging agent periodically reads and relays to the logging backend. The same principles also apply to side-car logging, where in Sidecar streaming, the sidecar writes to a log file, while a sidecar logging agent communicates with the backend directly.
Container logs live at `/var/log/pods/...`; `kubectl logs` just tails that node file (so a *streaming sidecar's* own logs aren't shown). Kubelet rotates at 10MB, keeps 5 files.
LogQL (Loki) selects by stream labels, then filters lines: `{job=ttest"} |= ërror"`. Loki requires unix *nanosecond* timestamps.

8.4 EFK stack

Also ELK, where Logstash replaces Fluentd
Elasticsearch: Search engine written in Java, works with indices to handle huge amounts of data, especially JSON documents.
Fluentd: Data Collection written in Ruby. Collects, filters and buffers logs from different sources, tries to store logging data as JSON. Typically runs as DaemonSet on each node and forwards Data to Elasticsearch.
Kibana: Similar to Grafana, Kibana can display data living in Elasticsearch.
EFK/ELK vs. Loki (PLG): Elasticsearch + full-text indexes the log *content* (powerful queries/analytics, mature, Kibana), - resource-hungry (CPU/RAM/-disk) & harder to scale. Loki + only indexes *labels*/metadata (cheap, lightweight, native Grafana & PromQL-like LogQL), - weaker arbitrary full-text search.

9 Service Mesh

9.1 Microservices

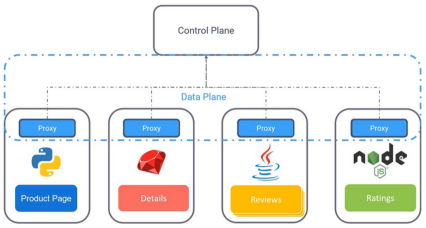
A microservice is a small, independent application that handles one specific business function and communicates with other services via APIs. This approach breaks large applications into smaller, loosely-coupled services that can be developed and deployed independently. Opposite of a Monolith.

9.2 Problems with Microservices

Communication: Services need to know all the endpoints of services they have to communicate with. A new microservice can result in many changes in existing services. **Security:** Communication in cluster is not secured, every service can talk to every other service in a cluster. Possible attackers have full access. **Monitoring:** Since services are spread out, monitoring can be tedious. Non Business logic must be added to each application, adds complexity.

9.3 Service Mesh w/ Sidecar Pattern

A Service Mesh uses Sidecars in all the Pods. The sidecars act as a proxy and also include additional functionality (Monitoring, Security, ...)



The two key components of a service mesh are: *Data Pane:* Consists of sidecar-proxies in pods which handle communication etc. (Example: Envoy) *Control Pane:* Manages and distributes configuration to data pane. (Example: Istiod)
Envoy proxies in each pod authenticate each other via **mTLS** (mutual TLS: client and server authenticate each other using TLS certificates)

9.4 Ambient Mesh

The Traffic Mesh with Sidecar pattern comes with some downsides: It is invasive, can lead to overallocation of resources per pod, breaks traffic.
Ambient mesh solves this by removing the sidecar proxies, rather providing one zTunnel per node. These zTunnels communicate with across nodes. They operate on Layer 4. zTunnels also authenticate each other using mTLS.
Additionally, optionally L7 waypoint proxies can be used for policy enforcements.

9.5 Gateway API & Ambient Routing

Enable ambient on a namespace: `kubectl label ns x istio.io/dataplane-mode=ambient` (classic sidecar mode uses the `istio-injection=enabled` label). zTunnel alone gives only L4 metrics; add a **waypoint** proxy for L7. In ambient the course routes via the K8 **Gateway API** (`HTTPRoute`) rather than a VirtualService. Two Deployments behind one Service get ≈50/50 by default (plain K8 load balancing); override with weights. Best practice: a `version` label per Deployment (routing + tracing).

```
# weighted canary (90/10) via HTTPRoute:
- backendRefs:
  - { name: frontend, port: 5000, weight: 90 }
  - { name: frontend-v2, port: 5000, weight: 10 }
# traffic mirror/shadow (copy to v2, response
      dropped):
- backendRefs:
  - { name: frontend, port: 5000 }
  filters:
  - type: RequestMirror
    requestMirror:
      backendRef: { name: frontend-v2, port: 5000
          }
```

istio-system pods: Istiod (control plane), Ingress Gateway (Envoy entrypo-

int), Prometheus, Grafana, Jaeger (tracing), Kiali (dashboard).
Kiali graph shapes: rectangle = application, triangle = Service, square = workload/pod.

9.6 Traffic Management

In istio, K8 ingress/egress are replaced by gateways, as they allow for more control. A VirtualService binds to a gateway and determines how traffic is routed to services. DestinationRules apply policies *after* routing: load balancing, connection pool, outlier detection (circuit breaking), TLS mode, and define *subsets* (e.g. versions) that VirtualServices route to.

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: my-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "example.com"
--
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: my-vs
spec:
  hosts:
  - "example.ch" # must match a gateway host
  gateways:
  - my-gateway
  http:
  - route:
    - destination:
        host: my-service
        weight: 99 # if a second destination is
              defined, weight can be used to
              split traffic accordingly
        port:
          number: 80
--
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: my-dr
spec:
  host: my-service
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
```

9.7 Service Resiliency

Istio allows for: **Fault Injection:** Simulate networking failures to test error-handling. **Timeouts:** Times out requests after a certain time to avoid queuing up a lot of requests. **Retries:** Configure services to retry if another service cannot be reached.
Istio also allows all the CD&D Deployment Strategies (see *General Definitions*)
Istio provides and exports Prometheus metrics per default.

9.8 Security

Authentication: Services use mTLS to authenticate each other.
Authorization: Using AuthorizationPolicies, we get Fine-grained access control using Role-based access control (RBAC).

```
# Namespace-wide mTLS enforcement using
      PeerAuthentication
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: production # Replace with your
        target namespace
spec:
  mtls:
    mode: STRICT
--
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: require-jwt
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin # applies to these pods in
            namespace foo
  action: ALLOW
  rules:
  - from:
    - source:
        namespaces: ["frontend"] # allows traffic
              from frontend NS
    to:
    - operation:
        methods: ["GET"]
        paths: ["/headers"] # allows traffic to
        path: "/api/.*"
```

10 Cilium & eBPF

10.1 eBPF

eBPF (extended Berkeley Packet Filter): runs sandboxed programs directly in the Linux kernel without changing kernel source or loading modules. Programs attach to hooks (syscalls, network events) and are verified before loading. Enables high-performance networking, observability and security at kernel level.

10.2 Cilium

A CNI (Container Network Interface) plugin built on eBPF. Replaces `kube-proxy`: instead of one iptables rule chain per Service, it programs eBPF maps in the kernel (faster, scales better).
Identity-based security: Cilium derives a security *identity* from a pod's labels (not from its IP). Policy is enforced on identities, so it survives pod rescheduling and IP changes.
Hubble: Cilium's observability layer; gives network flow visibility (service map, L3–L7 flows).

10.3 Network Policies

Declarative firewall rules for pod traffic. Plain `NetworkPolicy` works at L3/L4 (IP, port, protocol). `CiliumNetworkPolicy` adds L7 (HTTP method/path, DNS, gRPC, Kafka).
Default deny: as soon as a pod is selected by any policy, all non-matching traffic is dropped (whitelist model). Rules split into **ingress** (incoming) and **egress** (outgoing) and are *stateful*: return traffic of an allowed connection is permitted automatically.

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: allow-frontend
spec:
  endpointSelector:
    matchLabels:
      app: backend # applies to backend pods
  ingress:
  - fromEndpoints:
    - matchLabels:
        app: frontend # only frontend may connect
    toPorts:
    - ports:
      - port: "80"
        protocol: TCP
      rules:
        http: # L7 rule
        - method: GET
```

11 GitOps

GitOps uses Git repositories as the single source of truth for infrastructure and application configurations. Automated agents continuously monitor Git repos and sync changes to target environments automatically.
Principles: *declarative* (desired state lives in Git), *versioned & immutable* (Git history = audit log, revert = rollback), *pulled automatically* and *continuously reconciled* (agent detects and corrects drift).

11.1 Push- vs. Pull-Based

Push (classic CI/CD): the pipeline holds cluster credentials and pushes manifests into the cluster (`kubectl apply`). - credentials live outside the cluster, - no drift detection, - every new environment needs pipeline changes.
Pull (GitOps): an agent inside the cluster watches Git and pulls the desired state. + no external credentials / open firewall, + automatic drift correction, + scales to many clusters, - an agent must be installed per cluster.

11.2 Drift & Reconciliation

Drift = the live cluster state diverges from Git (e.g. a manual `kubectl edit`). The agent runs a *reconciliation loop*: continuously compare Git vs. live state and either report the difference or self-heal (sync/prune) back to the declared state.

11.3 Argo CD vs. Flux CD

Argo CD: platform-engineering tool with a web UI & dashboard. Wraps a deployment in an `Application` CRD (points at repo+path and a target cluster/namespace). `ApplicationSet` templates many Applications (multi-cluster, monorepo, per-env). Supports sync waves, health checks, manual or auto sync.
Flux CD: lightweight, CLI-first, built from the GitOps Toolkit controllers. Kustomize- & Helm-native (`Kustomization`, `HelmRelease` CRDs), no built-in UI. Strong on image automation & notifications.

11.4 Argo CD in Practice

```
kubectl create namespace argocd
kubectl apply -n argocd -f .../install.yaml
kubectl port-forward -n argocd svc/argocd-server
      8080:443
# initial admin password (user: admin):
kubectl -n argocd get secret argocd-initial-admin
      -secret \
  -o jsonpath="{.data.password}" | base64 -d
```

Destination for the local cluster: `https://kubernetes.default.svc`.
Gotcha: if a manifest sets `replicas` *and* an HPA is active, Argo CD and the HPA fight (revert vs rescale) → omit `replicas` from Git.
Secrets in GitOps: (1) *encrypted in Git* (Sealed Secrets, SOPS), or (2) *reference in Git* + fetch from an external store (External Secrets, Vault).
Access direction: push = runner needs access *into* the cluster; pull = the cluster reaches *out* to Git (nothing reaches in).

12 Site Reliability Engineering (SRE)

"Keep the site up, whatever it takes"
SL Indicators: A measurement (e.g. Response latency over a period of 10 min) **SL Objectives:** A Objective (e.g. 99.9% below 5ms) **SL Agreements:** A economic incentive (e.g. or less bonus)
Error Budget: 1 - SLO (e.g. 100% - 99.9% = 0.1%)

The four golden signals: latency, traffic, errors, and saturation.

12.1 Outage Handling

Goals: 1. Minimize impact (Automated, fast detection; good diagnostic information, practice remidation) 2. Prevent reoccurrence (handle event, write post-mortem)